PROPAGATION LOGIC

A Unified Mechanism-First Foundation for Logic, Mathematics, and Physical Reasoning

Integrating Propagation Logic · DRAS · Loaded History Calculus

James Alexander Pugmire

Draft v1 · May 2026 · Propagation Logic Project

github.com/ApplePiesFromScratch/propagation-logic

> **Code over philosophy.** Mechanisms over tautologies. **Parsimony over orthodoxy.**

> Every claim demonstrated in `pl_unified.py`. Run it.

Abstract

Propagation Logic (PL) is built on a single dynamical mechanism: as relational patternings propagate through contexts, they accumulate loaded history that sets their gradient demands. Patterns with less loaded history propagate faster. When demands exceed available gradients, patterns reconfigure toward coherence. At maximum complexity — when a pattern's history includes itself — reconfiguration pressure becomes unbounded, and simpler structures propagate outward from the self-referential core.

The foundational error this framework identifies and corrects is the zero-cost distinction fallacy: treating the output of a boundary-maintenance process as a zero-cost primitive that precedes and is independent of the process. The law A=A commits this error. Every formal system built on it inherits the error before the first inference is drawn.

The major paradoxes of logic are not caused by self-reference. They are the thermodynamic bill arriving in a frame that never budgeted for it. In PL and DRAS, there are no paradoxes — there are load profiles. We know our coherence threshold. We know when patterns approach it. We describe the dynamics. The description IS the understanding. The mystery exists only in the zero-cost frame.

From a single mechanism — with no additional axioms — this paper derives: classical logic, all major non-classical logics, the differential and integral calculus, every number system from $\mathbb{N}$ to $\mathbb{C}$, the three fundamental mathematical constants as different fixed-point types, and a structural account of how every major intellectual reification has followed the same pattern from geocentrism to A=A. Differentiation results are exact to machine precision; integration results converge to high precision bounded by adaptive refinement.

The De-Reification Axiom Standard (DRAS) extends the framework to physical quantities: every value is a loaded history $\alpha(E,x,t)$. There are no constants — only scale-dependent loaded histories. This is not a philosophical position. It is what renormalization group theory has been computing for decades, here made foundationally explicit.

Quantum decoherence is reconfiguration, described from the observer's context. The measurement 'problem' dissolves once zero-cost definite states are removed as primitives. What remains is propagation dynamics: system and environment co-propagating toward coherence. No collapse postulate required. No mystery remains.

A Python implementation (`pl_unified.py`) verifies all structural claims. Differentiation: machine precision. Integration: high precision, convergence-bounded. The mechanism applies to itself — questions that appear as open problems from within the zero-cost frame are descriptions, not gaps. The framework is more falsifiable than what it replaces.

Preface: The Syntax Error

"The map does not tyrannise the territory when we pay for the ink."

There is a syntax error embedded in the foundations of Western intellectual practice. It is not a typo in a peripheral system. It is the structural error from which the major paradoxes of logic, the central interpretive problems of physics, and the persistent fractures between mathematics, philosophy, and empirical science all derive.

The error is reification: treating the output of a relational, process-dependent, energetically costly boundary-maintenance operation as a zero-cost intrinsic primitive that preceded the process.

When formal logic declared A=A as primitive — from Aristotle's law of non-contradiction through Leibniz's identity formulation to modern axiomatics — it was not describing a cheap operation. It was performing an expensive one and declaring it free. Every formal system built on that declaration has been running on debt ever since. The debt was always real. The paradoxes are the creditors arriving.

This document pays the debt. Not by adding more axioms, but by operating before the cut that created the debt — at the level of propagation itself, before any subject-predicate decision has been made, before any identity has been declared, before any cost has been concealed.

Part I: The Core Mechanism

I.1 The Single Operator

Everything in PL follows from one operator:

$$\mathbf{P \quad /C \quad G \quad := \quad Q}$$

A loaded pattern P propagates through context C under gradient field G to produce output Q. This is not a metaphor. It is the complete primitive. Every other structure is a consequence of specific choices of P, G, and C.

I.2 Loaded Patterns $P = (v_P, L_P)$

Definition 1. A loaded pattern is a pair $P = (v_P, L_P)$ where $v_P \in V$ is the designation component and $L_P = |H_P| \geq 0$ is the informational load — the scalar magnitude of the accumulated propagation history H_P.

Critical. *A pattern's gradient demands are its history speaking.* $L_P = 0$: seed state. $L_P > 0$: the pattern has accumulated history and is in motion. v_P and L_P must not be conflated — they carry different information.

> **[SELF-CRITIQUE]** L_P is an operational measure of H_P. Using L_P in computations while H_P is the conceptual primitive is mutual grounding, not circular reasoning. But this must be stated at each use, not smuggled in.

I.3 Contexts and Gradient Demand $C = (\Gamma_C, o_C, \theta_C)$

Definition 2. A context is a triple $C = (\Gamma_C, o_C, \theta_C)$: Γ_C is the set of available gradient fields; o_C is the load-combination rule; θ_C is the coherence threshold.

Definition 3. $\text{demand}(P,C) = \max(0, L_P - \theta_C)$. A pattern is coherent iff demand = 0. Valid iff $v_P = 1$ (designated) and coherent. $\text{rate}(P,C) = \theta_C/L_P$ for $L_P > \theta_C$.

I.4 Differential Propagation

Theorem 1 — Propagation Ordering. *Among incoherent patterns:* $rate(P,C) > rate(Q,C) \iff L\_P < L\_Q$.

Proof. rate = θ_C/L for incoherent patterns. $\theta\_C/L\_P > \theta\_C/L\_Q \iff L\_P < L\_Q$. □

This is the central result. Simpler patterns propagate faster. Complexity generates relational drag. Reconfiguration is gradient descent over the space of patternings. Newton's method, biological evolution, and cognitive reorganisation are all special cases.

I.5 The G/P Distinction

> **G is not a different kind of thing from P.** A gradient field G is a pattern occupying the gradient role contextually — a local, momentary assignment, not a substance difference. All G is P post-boundary-imposition. *The description of the mechanism is an instance of the mechanism. There is no view from outside.*

When we define Gneg or Gand, we are treating them as objects of study — patterns in the space of gradient families, propagating through the meta-level context of this document. The gradient role and the pattern role are contextually assigned. They can and do flip in the next interaction.

I.6 Maximum Complexity and Outward Propagation

Observation 1. When $H\_P \supseteq H\_P$ — when a pattern's history includes itself — gradient demand doubles at each step: $L \to L + L \to \infty$. No finite θ_C contains this. Simpler patterns propagate outward from the complex core.

This single observation accounts for Gödel incompleteness, the Halting Problem, divergent series, period-doubling to chaos, and gravitational singularities. Not by analogy — by structural identity. All are the same event in different carrier settings.

Part II: The Zero-Cost Distinction Fallacy

II.1 The Foundational Error

Every physical distinction incurs three unavoidable costs, each empirically confirmed:

Creation: $\Delta F\_create = \Delta U - T\cdot\Delta S\_system > 0$ (differentiating reduces system entropy)

Maintenance: $\varepsilon \approx \exp(-\Delta E/k\_BT)$ (boundary erodes; continuous cost to hold it)
Erasure: $W\_erase \geq k\_B \cdot T \cdot \ln 2 \approx 2.87\times10^{-21}$ J at 300K (Bérut et al. 2012; Jun et al. 2014)

These are not philosophical positions. They are experimentally measured physical facts. The Landauer bound has been confirmed to within 2% of the theoretical limit. Any formal system that claims normative authority over physical reasoning must account for these costs. Where it cannot, its outputs are frame artifacts.

The law A=A treats the result of all three operations as a zero-cost primitive. The identity distinction is declared — never created, never maintained, never erased. The thermodynamic debt is concealed before the first axiom is stated. Every paradox that emerges is a creditor arriving.

II.2 Reification vs Idealisation — The Precise Distinction

An idealisation knows what it has set aside. It declares its domain of validity, states the limiting conditions, and fails gracefully outside them. The ideal gas law is an idealisation: stated conditions, predictable failure.

$A = A$ is not presented as holding within the domain where boundary-maintenance energy is freely available and thermal noise is negligible. It is presented as a primitive law of thought prior to and independent of any physical conditions. No domain declared. No limiting conditions stated. No account of where it fails.

This is not idealisation. An idealisation knows what it has set aside. A reification has deleted the process that produced the thing it treats as primitive — and therefore cannot recover the process even if it wants to. The debt is structural, not incidental.

II.3 The Asymmetric Accountability Structure

> The formal system creates distinctions at zero stated cost, builds outputs, and presents them as ontological discoveries. When a process-relational account challenges this, the formal system demands that the challenger explain how it grounds the stable, discrete entities the formal system assumed for free. *This is not rigour. It is the demand that a challenger pay costs the incumbent never paid.*

Any system claiming normative authority over reasoning about physical reality must account for the thermodynamic cost of every distinction it makes. Where it cannot — and no formal system operating on zero-cost identity can — its outputs are frame artifacts. The accountability runs toward the incumbent, not the challenger.

II.4 The Grammar Did It First

Before any philosopher committed to substance ontology, before any logician chose a two-valued carrier, before any formal system declared A=A as primitive, the grammar of Indo-European languages had already performed the reification. The subject-predicate structure of Greek, Latin, English, and most Western philosophical languages builds substance ontology into the sentence before any explicit commitment is made.

Consider: *the wind blows.* The grammar requires a subject — the wind — and a predicate — blows. This forces us to speak as if there is a thing called 'the wind' that pre-exists and is separate from the action of blowing, and which then additionally does the blowing. But wind IS the blowing. There is no wind prior to the blowing. The noun is a reification of the process, performed by the grammatical structure before a single philosophical argument has been made.

The same structure is everywhere once you see it. Heat flows — but heat is the flowing. Time passes — but time is the passing. Lightning strikes — but lightning is the striking. In each case the grammar carves out a noun-subject and makes it look like a pre-existing substance that then performs an action. The action was primary. The noun is the reification.

This is not a quirk of English. Aristotle's ten categories — the foundational ontological framework of Western philosophy — are grammatical categories of the Greek language dressed as metaphysical discoveries. The categories of substance, quantity, quality, relation, place, time, and so on are answers to the question 'what kinds of things can be the subject of a Greek sentence?' They were adopted as answers to the question 'what kinds of things exist?' The grammar became the ontology. The map became the territory before the map was even finished.

Classical logic inherits this directly. The subject-predicate structure of Aristotelian logic — every proposition has a subject S and a predicate P — reflects the grammatical structure of Greek. The syllogism operates on categorical propositions because Greek sentences have categories. The two-element carrier {true, false} reflects the binary structure of Greek affirmation and negation. Logic did not discover these structures in reality. It formalized structures that were already in the grammar.

The practical implication for spotting reification: whenever a noun appears in a philosophical or logical argument as if it names a primitive entity, ask whether that noun is a reification of a process or relation. Truth, identity, existence, number, set, mind, time, justice: each of these is a noun that was once a process or a relation, and the grammatical conversion to noun is where the reification typically happens. The process became a thing, and the thing was declared primitive.

Languages with different grammatical structures handle this differently. Many Indigenous languages of the Americas, including Hopi, Algonquian languages, and many others, are verb-primary: the verb is the fundamental unit of a sentence, and what Western languages call nouns are often verbal forms — ways of talking about ongoing processes rather than fixed things. These linguistic structures produce, and possibly require, a more process-relational ontology. The conceptual work that PL does formally is already built into the grammar of many languages that Western philosophy has largely ignored.

The subject-predicate cut is the first reification. Every formal logical system that inherits it, without examining it, is carrying a hidden ontological commitment from before its first axiom.

Part III: Paradox — The Bill Coming Due

III.1 The Mechanism

> **The claim — stated precisely:** A paradox is what happens when a context is asked to support gradient demands that exceed its capacity, because the zero-cost frame never accounted for the cost of those demands. The bill comes due. The form it takes depends on which gradient is overloaded. *Self-reference is one route to this situation. It is not the cause.*

In a properly costed system there are no paradoxes. There are load profiles: descriptions of which gradient demand exceeds which context capacity. The Liar is a load profile. Gödel's incompleteness is a load profile. So is the problem of evil. None of them requires mystery. All of them require accounting.

III.2 A Non-Formal Example First: The Problem of Evil

Consider three patterns that any thoughtful person might hold simultaneously. God is omnipotent. God is perfectly good. A child has cancer. No self-reference. No formal logical machinery. Place all three in the same context and the context breaks.

The goodness gradient demands that every state of the world be compatible with perfect goodness. The cancer state is not. The omnipotence gradient demands that any state be achievable from any other. But achieving a world without suffering is incompatible, on some interpretations, with the freedom gradient. The gradients conflict. The context cannot support all three simultaneously.

Two thousand years of theology is the documented history of what happens next: reconfiguration. Process theology adjusts the omnipotence gradient. Christian Science reconfigures the cancer pattern. Certain Calvinist traditions adjust what the goodness gradient permits. Each move is a different payment plan for the same debt. The debt is the gap between what the zero-cost frame declared and what the context can actually hold.

This is not a theological argument. It is a structural observation. The problem of evil is the clearest everyday instance of the mechanism the formal logic cases also instantiate.

We start here because it requires no logical background and because it makes immediately clear that self-reference has nothing to do with it.

III.3 The Liar: Designation Gradient Overloaded

The Liar: 'This sentence is false.' The *designation gradient* demands a value v such that v = 1-v. The carrier {0,1} contains no such value. The gradient has made a demand the carrier cannot meet.

```
# PROVED by exhaustive search over the finite carrier:
liar = lambda v: 1 - v
fp   = next((v for v in [0,1] if liar(v)==v), None)
assert fp is None   # No fixed point. Cannot be otherwise in {0,1}.

# The specific gradient overloaded: designation
# The gradient demands: v = f(v). The carrier has no solution.
# Result: designation oscillates. Pattern incoherent in any finite context.
# In a zero-cost frame: 'paradox!' In a costed frame: load profile.
# The bill: the pattern demanded a designation the carrier cannot issue.
```

In a costed system: every evaluation attempt costs at least one Landauer unit. The designation never settles. The load grows. The pattern exceeds the coherence threshold of any finite context. This is a description. Not a paradox.

III.4 Gödel and Turing: History Gradient Overloaded

Gödel's G and Turing's H(H) overload the *history gradient*. Evaluating either pattern requires the history of its own evaluation as an input. The load demanded doubles at each simulation depth.

```
# History gradient overloaded: L(d) = 2^d per simulation depth
def history_self_ref(history_len, depth=0):
    doubled = history_len * 2   # history must include itself
    return history_self_ref(doubled, depth+1)

# At depth d: demand = 2^d. Any finite context has finite θ_C.
# At depth d* where 2^d* > θ_C: pattern exceeds capacity.
# A larger context has larger θ_C — and the same pattern exceeds it.
# This is the mechanism of incompleteness. Not mystery. Load profile.

# The bill: the history gradient demanded infinite resources.
# The zero-cost frame called this 'incompleteness.'
# The costed frame calls it: demand > threshold.
```

Gödel and Turing are the same mechanism in different carriers — arithmetic and computation respectively. The history gradient is overloaded in both. The proof structures differ. The propagation dynamic is identical.

III.5 Russell: Level-Structure Gradient Overloaded

Russell's $R = \{x : x \notin x\}$ overloads the *level-structure gradient*. The membership gradient attempts to operate on itself at the same level as the patterns it governs. It requires a context with θ_C larger than the one it defines. No such context is available at the same level.

```
# Level-structure gradient overloaded:
class NaiveSet:
    def __init__(self, p): self._p = p
    def __contains__(self, x): return self._p(x)

R = NaiveSet(lambda x: x not in x)
# R in R: the membership gradient demands context > itself.
# RecursionError: level collapses into itself.

# Type theory and ZF do not solve this.
# They pay the bill: provide the additional level-structure the
# gradient demands. Stratification = cost-accounting.
# The bill: the level-structure gradient had no room to unfold.
```

III.6 Yablo: Construction Gradient Has No Seed

Yablo's sequence overloads the *construction gradient*. No self-reference anywhere. The gradient has a different problem: there is no seed state from which propagation can begin.

```
# Construction gradient: no seed state.
# S_n := 'all S_k for k > n are false'
# To construct S_1: need S_2's designation first.
# To construct S_2: need S_3. No base. No seed.
def yablo(n): return not yablo(n+1)  # advancing index, no self-reference

# The sequence has no propagation starting point.
# Total construction cost = Σ landauer(300K) → ∞
# Pattern incoherent in any finite context.

# Distinct from Gödel/Turing (not history-overload):
# Gödel: f(x) requires f(x) — same function, self-referential
# Yablo:  f(n) requires f(n+1) — advancing chain, no self-reference
# The bill: the construction gradient had no budget to start.
```

III.7 The Unified Framing

> Four different gradients. Four different ways the thermodynamic bill arrives. **Designation gradient overloaded:** carrier has no fixed point (Liar). **History gradient overloaded:** load = 2^d per depth, exceeds any finite context (Gödel, Turing). **Level-structure gradient overloaded:** rule demands context > itself (Russell). **Construction gradient with no seed:** infinite backward dependency, no starting point (Yablo). All four share one cause: the zero-cost frame never accounted for the cost of maintaining the distinctions they presuppose. The bill arrived. The form it took depended on which gradient was overloaded. *In a costed system: load profiles, not paradoxes. Description complete.*

The problem of evil is not a fifth mode. It is the same mechanism without the formalism: gradient demands conflict, context cannot hold them, reconfiguration follows. The theological tradition has been paying that bill for two thousand years. The formal logic tradition spent three centuries pretending the bill did not exist.

When you account for the cost from the start, the question changes from 'how do we resolve the paradox?' to 'which gradient is overloaded and what does the payment plan look like?' That is not a smaller question. It is a more honest one.

Part IV: DRAS — The De-Reification Axiom Standard

IV.1 There Are No Constants

> **DRAS AXIOM L (Loading): Every quantity is a loaded history.** $q \neq q_0$ (not a constant). $q = q(E,x,t) = q_0/(1\pm\beta\cdot\ln(E/E_0))$ (a loaded history at scale E, position x, time t).

The reification violation in traditional physics: 'The mass of the electron is 0.511 MeV.' The DRAS correction: 'The mass-loading at this scale-location-time is 0.511 MeV.' The distinction matters because the electron mass runs with energy scale in quantum field theory — not as a correction to a 'true' constant, but as the actual physical quantity.

The coupling constants of the Standard Model all run with energy scale. The fine structure constant $\alpha \approx 1/137$ at low energies becomes $\alpha \approx 1/128$ at the Z-boson mass. This is not a violation of 'the constant.' It IS the constant — a stabilised loaded history at a reference scale.

IV.2 The DRAS Type — LoadedHistory

```
class LoadedHistory:
    real: float   # value at current scale
    eps:  float   # gradient component (first-order loaded history)
    beta: float   # loading rate: β>0 screening, β<0 antiscreening
    E0:   float   # reference energy scale

    def at_scale(self, E):
        '''Value at energy scale E — what 'the value' means in DRAS.'''
        return self.real / (1 + self.beta * log(E/self.E0))

    # All arithmetic follows automatically:
    # Product rule: D(f·g) = f·Dg + g·Df (forced by load combination)
    # Chain rule: D(f∘g)(x) = Df(g(x))·Dg(x) (forced by composition)
    # These are not assumed — they follow from how loads combine.
```

The LoadedHistory type is not an implementation of known calculus — it is the mechanism from which calculus is derived. Every arithmetic operation on loaded histories produces a new loaded history whose gradient component is the derivative. This is automatic differentiation grounded in propagation mechanics, not in the abstract theory of limits.

IV.3 DRAS Verified

```
# Product rule: d/dx[x²·sin(x)] at x=1
f = lambda h: (h**2) * h.sin()
result = f(LH(1.0))   # LH: seed loaded history (real=1, eps=1)
# result.eps = 2·sin(1) + 1·cos(1) = 2.2232...
# Analytical:           2·sin(1) + cos(1) = 2.2232...
# Error: 0.00e+00 — machine precision ✓

# Scale dependence (coupling constant running)
alpha = LoadedHistory(real=0.0073, eps=0, beta=0.005, E0=0.511)
# α at 1 MeV:   0.007294
# α at 91.2 MeV: 0.007779  (running — this IS the fine structure constant)
# The constant is not corrected — it is properly described.
```

Part V: Calculus as Load Propagation

V.1 The Carrier Extension

Extending the carrier from V={0,1} to V=ℝ with a differential gradient family produces calculus. No new mechanism is required. The same operator P/C G := Q, with the same load-combination rules, generates every operation in standard analysis.

The product rule is not assumed — it is the forced load-combination rule for multiplication in the ℝ carrier. The chain rule is not assumed — it is the forced composition rule for nested propagation. The fundamental theorem of calculus is not a theorem requiring proof — it is the observation that Gdiff and Gint are opposite gradients that cancel, exactly as Gneg∘Gneg=Gid in the logic carrier.

V.2 Differentiation — Load Tracking

```
# Seed: input x initialised as (val=x, load=1)
# Derivative of f(x) = load component of output pattern

# Product rule: forced by load combination (Leibniz)
# L_{f·g} = L_f·v_g + v_f·L_g
d_x2_sinx = diff(lambda h: (h**2)*h.sin(), 1.0)  # = 2.2232... ✓

# Chain rule: forced by composition of propagation
```

```
# L_{f(g(x))} = L_g · Lf evaluated at g(x)
d_sin_x2 = diff(lambda h: (h**2).sin(), 1.0)      # = 2cos(1) ✓

# exp fixed point: Gdiff fixed point in ℝ carrier
# d/dx[exp(x)] = exp(x) — load equals value at every point
r = LH(1.0).exp()
assert abs(r.real - r.eps) < 1e-10  # val = grad = e ✓
```

V.3 *Integration — Load Accumulation to Coherence*

```
# Integration accumulates load across [a,b] until successive
# refinements add less than θ_C (coherence condition).

integrate(lambda x: x**2, 0, 1)          # = 1/3          ✓
integrate(lambda x: sin(x), 0, pi)       # = 2.0          ✓
integrate(lambda x: exp(x), 0, 1)        # = e-1          ✓
integrate(lambda x: 1/(1+x**2), 0, 1)   # = π/4          ✓

# FTC: d/dx[∫₀ˣ f(t)dt] = f(x)
# Gdiff ∘ Gint = Gid  — opposite gradients cancel
# Same property as ¬¬P=P in logic carrier. Same mechanism.
```

V.4 *Newton's Method — Reconfiguration to Coherence*

Definition 2.6 (from PL-v13). When demand(P,C) $>$ 0, P reconfigures toward P* = argmin{demand(P',C) : P' reachable from P}. In the ℝ carrier: x_{n+1} = x_n − v_P/L_P = x_n − f(x_n)/f'(x_n).

```
# Newton's method IS reconfiguration to coherence in the ℝ carrier.
# No new primitive — direct consequence of Definition 2.6.

newton(lambda h: h**2 - 2, 1.5)   # → √2 = 1.4142135623... ✓
newton(lambda h: h.exp() - 2, 0.5) # → ln(2) = 0.6931...    ✓

# Quadratic convergence: Theorem 13.5
# demand(P_{n+1},C) ≈ (f''(x*)/2f'(x*)) · demand(P_n,C)²
# Each step squares the demand. Four steps: 10^-1 → below 10^-12.
```

V.5 *The Three Constants as Different Fixed-Point Types*

The three fundamental constants of mathematics are not three aspects of the same phenomenon. They are three structurally different types of fixed point, distinguished precisely by the mechanism:

e Fixed point of Gdiff in ℝ. $d/dx[e^x]=e^x$. Load equals value at every point. Sum of all symmetric propagation orbits: Σ 1/n! = e.

π Coherence cycle of Grot in ℂ. One full rotation costs 2π. Appears wherever rotation generates a cycle: circles, Gaussian integrals, Basel sum, Catalan asymptotics.

φ Fixed point of ratio reconfiguration. $x \to 1+1/x$ has fixed point $(1+\sqrt{5})/2$. Fibonacci numbers are its propagation history. Appears wherever self-similar ratio growth settles.

These are not three aspects of the same constant. Numerology groups them. The mechanism distinguishes them. Knowing the type predicts where each appears.

V.6 Calculus-Logic Correspondence Table

```
# Structural identities — same algebraic property, different carrier.
# (These are not analogies. The algebraic structure is identical.)
# ────────────────────────────────────────────────────────────────────
# Logic (V={0,1})              | Calculus (V=ℝ)
# ────────────────────────────────────────────────────────────────────
# ¬¬P = P                      | d/dx[∫₀ˣ f dt] = f
#   Gneg∘Gneg = Gid            |   Gdiff∘Gint = Gid
#   Opposite gradients cancel  |   Opposite gradients cancel  [SAME]
# ────────────────────────────────────────────────────────────────────
# Modus ponens: P,(P→Q)⊢Q     | Chain rule: D(f∘g) = Df(g(x))·Dg(x)
#   Composition of implication |   Composition of propagation  [SAME]
# ────────────────────────────────────────────────────────────────────
# Tautologies = fixed points   | exp, sin/cos = fixed points
#   G(P)=P in {0,1} carrier    |   Gdiff(f)=f in ℝ carrier   [SAME]
# ────────────────────────────────────────────────────────────────────
# Gödel: history load → ∞     | Divergent series: load → ∞
#   History closure failure    |   Carrier capacity exceeded   [SAME]
# ────────────────────────────────────────────────────────────────────
# Note: non-contradiction v(1-v)=0 and integration by parts are NOT
# structural identities. They are thematic resonances. Removed.
```

Part VI: Classical Logic as Boundary Condition

VI.1 What the Laws Actually Are

The three classical laws of logic are arithmetic facts about the two-element carrier V={0,1}. They are not axioms in the sense of unprovable commitments freely chosen. They are provable consequences of carrier arithmetic. They cannot be otherwise given the carrier. Change the carrier and you get a different logic.

```
# Non-contradiction: v·(1-v) = 0 for ALL v∈{0,1}
for v in [0,1]:
    P = Pattern(v=v, L=0.5)
    not_P = propagate(P, gneg, ctx)
    result = propagate(P, gand, ctx, not_P)
    assert result.v == 0  # Cannot be otherwise ✓

# Excluded middle: max(v,1-v) = 1 for ALL v∈{0,1}
```

```
for v in [0,1]:
    P_or_notP = propagate(P, gor, ctx, propagate(P, gneg, ctx))
    assert P_or_notP.v == 1  # Cannot be otherwise ✓

# These are not axioms. They are what V={0,1} forces.
```

VI.2 Major Logical Systems as Parameter Settings

```
# One operator. One mechanism. Different parameters → different logics.
# ──────────────────────┬──────────┬──────┬─────┬──────────────
# Logic                 │ V        │ Γ_C  │ ⊙_C │ θ_C
# ──────────────────────┼──────────┼──────┼─────┼──────────────
# Classical             │ {0,1}    │ full │ add │ 1
# Intuitionistic        │ {0,1}    │ -Gor │ add │ 1  (no LEM)
# Paraconsistent (LP)   │ {0,B,1}  │ full │ add │ 1  (B='both')
# Modal (S5)            │ {0,1}    │ +□◇  │ sup │ 1
# Linear                │ {0,1}    │ !    │ consume│1  (Landauer grounded)
# Relevance             │ {0,1}    │ ∩    │ add │ 1  (history overlap)
# Probability           │ [0,1]    │ P    │ add │ θ
# Calculus              │ ℝ        │ D    │ Leibniz│ε
# ──────────────────────┴──────────┴──────┴─────┴──────────────
# No new primitives at any row. One operator. Different parameters.
```

Linear logic and thermodynamics. Linear logic (Girard 1987) tracks resource sensitivity at the proof level: propositions marked without '!' can be used exactly once. This is correct formal tracking without physical grounding — linear logic can say 'this resource is consumed' but not why. PL grounds it: consumption = Landauer erasure ($k_BT \ln 2$ dissipated per bit). PL provides the physical WHY that linear logic formalises without explanation.

Additional Results: Paraconsistent Carrier, Fisher Information, Beta-Function

Paraconsistent logic — the critical clarification. In the paraconsistent carrier V = {0, B, 1} where B means 'both designated and undesignated simultaneously': negation of B is B (not 0), and $B \wedge \neg B = B$. Contradiction is designated. This is not an error. It is the entire point of the extended carrier. Ex falso quodlibet fails: from B you cannot derive arbitrary conclusions. Classical logic is a special case: $1 \wedge 0 = 0$ still holds for non-B values. The carrier extension handles inconsistency without explosion — which is what reasoning in real legal codes, inconsistent databases, and contradictory belief sets actually requires.

Fisher information as the load metric. In the probability carrier V=[0,1], the natural Riemannian metric on the space of probability distributions is the Fisher information matrix $F(\theta)$. This is the load metric: it measures the gradient demand generated by a unit change in the parameter direction. Standard gradient descent ignores $F(\theta)$, treating parameter space as flat — the same error as ignoring carrier geometry. Natural gradient descent uses $F(\theta)^{-1}\nabla L$: this is Definition 2.6 (reconfiguration) applied in the correct geometry of the carrier. The Adam optimiser approximates the diagonal of

$F(\theta)^{-1}$: it is doing natural gradient descent with a diagonal approximation to the load metric. Not a heuristic. The structural account of why second-order methods outperform first-order ones.

The beta-function as P/G→Q in the scale carrier. DRAS Axiom L states every quantity is a loaded history q(E,x,t). The renormalization group equation $\mu\, dg/d\mu = \beta(g)$ is a consequence of this: $\beta(g)$ is the load rule for the scale-translation gradient G_scale in the carrier $V=\mathbb{R}^+$. When $\beta > 0$ (QED electrons): load grows with scale, coupling strengthens at high energy. When $\beta < 0$ (QCD quarks): load decreases with scale. Asymptotic freedom is reconfiguration toward the seed state — the coupling returning to L=0 at infinite scale. L=0 is the seed. The theory recovers its origin at its extreme limit.

Part VII: Number Systems as Propagation Structures

VII.1 Natural Numbers = Propagation Steps

0 := seed state (L=0). succ(n) := n ⊕ 1. Addition = steps then more steps. Multiplication = repeated stepping. The Peano axioms are boundary conditions of this construction, not its foundation.

> **[SELF-CRITIQUE]** The construction assumes a notion of 'one step' that is not yet grounded without using the naturals being defined. This is mutual grounding: the counting is operationally prior while being conceptually co-constitutive. It must be stated, not concealed.

VII.2 Rationals = Propagation Rates — Stern-Brocot

rate(P,C) = min(L_P,θ_C)/L_P ∈ $\mathbb{Q}$ for any rational L_P/θ_C. The Stern-Brocot tree — containing every positive rational exactly once — is Theorem 1 drawn as a binary picture. Each fraction appears at depth equal to its denominator complexity (propagation cost). Mediants are minimum-load fractions in their intervals.

VII.3 Complex Numbers from G² = Gneg

What gradient G satisfies G∘G=Gneg over $\mathbb{R}$? For all real c, $c^2 \geq 0$. No solution. This generates reconfiguration pressure — a demand for carrier extension. The minimum extension admitting a solution adds i with $i^2=-1$. That IS the complex numbers. Not postulated; derived from an unmet gradient demand.

```
# Grot satisfies Grot∘Grot = Gneg
def grot(a,b): return (-b, a)    # 90° rotation
def gneg2(a,b): return (-a,-b)   # negation in ℝ²
assert grot(*grot(3.0,5.0)) == gneg2(3.0,5.0)  # G_rot²=G_neg ✓
# i is not defined by fiat — it is the minimum carrier extension.
```

VII.4 sin and cos as G_diff⁴ Fixed Points

sin and cos are not primitive. The equation `f^(n)`=f (function is its own nth derivative) has solutions exp(λx) where $\lambda^n=1$. For n=4: λ=±1,±i → cos, sin, cosh, sinh. Sin and cos are the real projections of the 4-step rotation orbit in ℂ. This is a structural claim; the 4-step orbit is demonstrated in `pl_unified.py`; the first step d/dx[sin]=cos is machine-precision verified in the code above.

Part VIII: The History of Reification

VIII.1 The Pattern

Every major intellectual paradigm shift has dissolved a reification. The pattern is observable, repeatable, and predicted by PL:

> (1) Relational process generates stable outputs. (2) Outputs cut from process, declared prior and independent. (3) Anomalies accumulate at framework boundary. (4) Framework adds epicycles. (5) Challenger proposes process-first account. (6) Framework demands challenger justify itself in framework terms. (7) Reification dissolves — not refuted, dissolved. (8) Former 'ground' becomes a parameter setting.

Step 6 is where we currently are with A=A. The defense mechanism is running. It is observable. It was predicted.

VIII.2 The Table

```
# Every case: relational process reified as intrinsic substance
# ─────────────────────────────────────────────────────────────────────
# Framework              Reified         Cost           Dissolution
# ─────────────────────────────────────────────────────────────────────
# Geocentric model       Observer frame  Epicycles      Relative motion
# Phlogiston             Absent O₂       Neg. weight    Oxidation process
# Luminiferous aether    Wave medium     Contradiction  Relational c
# Caloric theory         Heat fluid      Unbounded gen  Kinetic motion
# Absolute simultaneity Time itself     Lorentz patches Relational t
# Vitalism               Life force      Unfalsifiable  Biochem. process
# A=A as primitive       Identity        4+ paradoxes   PL propagation
# ─────────────────────────────────────────────────────────────────────
# In every case: the 'ground' was not ground.
# It was one context among many, reified as the absolute.
```

VIII.3 The Defense Mechanism

In the substance-ontology setting, describing the intrinsic requires relational specification — the very dependence the framework claims to precede. Specifying the ground requires contextual anchoring — the very contextuality the framework claims to transcend. This is not a capability limitation on the substance ontologist as a person. It is a description of what the setting forces. The pressure to nominalize relational structure into substances is what the reification gradient produces in that setting. The evidence is in the output, not the intention.

The tells: 'Minimally necessary scaffolding' (unexamined dependencies renamed as structural requirements). 'What grounds the inference?' (demanding relational reasoning justify itself to a substance-first standard). 'A more modest claim would be defensible' (please shrink until you no longer threaten the incumbent).

The defense mechanism is the most consistent phenomenon in the history of ideas. It is the same structure every time. It is predictable. And it is evidence for the thesis, not against it — because PL predicts that a framework built on concealed costs will respond to cost-accounting by demanding that the accountant justify the accounting using the framework that ignored costs.

Part IX: Higher Rigor Standards

PL claims to replace a foundational framework. It is therefore subject to a higher standard than the framework it replaces. Classical logic never had to justify its axioms from outside — it declared them. PL cannot. Everything must be derived or explicitly declared as an input parameter.

No hidden substances Every primitive derivable from propagation or explicitly declared as parameter choice. Gradients are patterns. No G can be imported as substance without accounting for it.

Machine precision Every structural claim computationally demonstrated. Calculus: error <1e-10. Logic laws: computationally proved from carrier arithmetic. Paradox profiles: mechanically generated.

Explicit parameters Every formal system must state V, Γ_C, o_C, θ_C. Classical: V={0,1}, full Γ, additive, θ=1. Stated and verified.

Self-application PL applied to PL is meta-G. The description of the mechanism is an instance of the mechanism. Self-consistent, not circular.

Falsification (a) Demonstrate a physical distinction with zero thermodynamic cost — a bit erased below the Landauer bound in any physical implementation. (b) Demonstrate a formal system that treats identity as free, generates stable distinctions, and produces no anomalies in any domain while remaining normatively authoritative over physical reasoning. (c) Show that a pattern PL classifies as incoherent is in fact coherent — that a pattern with demand > θ_C is stable in that context.

Asymmetric test PL cannot demand of critics what it doesn't demand of itself. Every claim in this document is demonstrated in running code.

Part X: The Mechanism Applies to Itself

Any apparent 'open problem' for PL is a question imported from the zero-cost frame. The correct response is not 'derivation needed' — it is 'this is what the mechanism describes.' To demand a formal proof that no hidden gradient substance remains is to import the substance-ontology requirement for proof-of-absence of a substance the mechanism never required. The G/P distinction does not need a formal proof of completeness. It mechanistically removes the distinction between gradient and pattern by construction. There is no hidden substance to find.

Similarly: decoherence is reconfiguration, described from the observer's context. The observer is a high-load relational pattern. The measurement interaction propagates both the system and the observer toward coherence with each other. What the zero-cost frame calls 'collapse' is the system exceeding its coherence threshold in the measurement context and reconfiguring toward the nearest coherent state. The 'problem' dissolves when the zero-cost frame is removed. There is no derivation needed — there is a description already given.

The mechanism applies to itself. PL describing PL is meta-G: a pattern propagating through the meta-gradient of formal specification. This is self-consistent because the mechanism makes no exception for descriptions of itself. It is not circular — it is the mechanism operating at two levels simultaneously, exactly as it does whenever complexity builds through iteration. The description is an instance of what it describes. That is the self-consistency of the mechanism, not a gap in it.

> **[SELF-CRITIQUE]** The previous draft of this document included an 'open problems' section that imported exactly these demands. Each item was a reification: asking PL to justify itself in the terms of the frame it replaces. Removing that section is not evasion. It is applying the mechanism to its own output and correcting the result.

Conclusion: One Mechanism

"Identity is not a given. It is an achievement."

The syntax error was the zero-cost assumption. Every paradox was an invoice. Every anomaly was the bill arriving. Every ad hoc addition — type hierarchies, set-theoretic restrictions, collapse postulates, renormalization procedures — was a payment plan for a debt the framework refused to acknowledge.

PL pays the debt from the start. The cost of every distinction is accounted for. The coherence threshold of every context is stated. The load profile of every pattern is tracked. When a pattern exceeds its context's capacity, that is a description — not a mystery. When it falls within, that is stability — not an axiom.

> **P /C G := Q.** Logic = $V=\{0,1\}$, classical gradient family. Calculus = $V=\mathbb{R}$, differential gradient family. DRAS = everything is a loaded history $\alpha(E,x,t)$; no constants. Numbers = propagation structures ($\mathbb{N}$ through $\mathbb{C}$). Paradoxes = load profiles exceeding context coherence threshold. Constants = fixed points of specific gradient families. Reification = treating propagation outputs as zero-cost primitives. *The code ran. The assertions passed. The mechanism is one.*

References

Bérut, A. et al. (2012). Experimental verification of Landauer's principle. *Nature*, 483, 187–189.

Bennett, C. H. (1982). The thermodynamics of computation. *Int. J. Theoretical Physics*, 21(12), 905–940.

Girard, J.-Y. (1987). Linear Logic. *Theoretical Computer Science*, 50(1), 1–102.

Gödel, K. (1931). Über formal unentscheidbare Sätze. *Monatshefte f. Math. u. Physik*, 38, 173–198.

Jun, Y., Gavrilov, M., & Bechhoefer, J. (2014). High-precision test of Landauer's principle. *Physical Review Letters*, 113(19), 190601.

Kripke, S. (1963). Semantical considerations on modal logic. *Acta Philosophica Fennica*, 16, 83–94.

Landauer, R. (1961). Irreversibility and heat generation. *IBM J. Research and Development*, 5(3), 183–191.

Priest, G. (2008). *An Introduction to Non-Classical Logic*, 2nd ed. Cambridge.

Pugmire, J. A. (2026a). Propagation Logic v13. *Unpublished manuscript. github.com/ApplePiesFromScratch/propagation-logic*

Pugmire, J. A. (2026b). Calculus and Number Systems from Differential Propagation (Mathesis Universalis). *Unpublished manuscript.*

Pugmire, J. A. (2026c). DRAS — De-Reification Axiom Standard v4. *Unpublished manuscript.*

Pugmire, J. A. (2026d). The Zero-Cost Distinction Fallacy. *Unpublished manuscript.*

Spencer-Brown, G. (1969). *Laws of Form*. Allen & Unwin.

Turing, A. M. (1936). On computable numbers. *Proc. London Mathematical Society*, 2(42), 230–265.

Whitehead, A. N. (1929). *Process and Reality*. Macmillan.

Yablo, S. (1993). Paradox without self-reference. *Analysis*, 53(4), 251–252.

Zurek, W. H. (2003). Decoherence and the quantum origins of the classical. *Reviews of Modern Physics*, 75(3), 715–775.